Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An autonomous Institution affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Name:	Sabarithan P	Roll No:	3122235001108
Due Date	27-01-2026		

Experiment 2: Binary Classification using Naïve Bayes and K-Nearest Neighbors

1. Aim & Objective

To implement Naïve Bayes and K-Nearest Neighbors (KNN) classifiers for a binary classification problem, evaluate them using multiple performance metrics, visualize model behavior, and analyze overfitting, underfitting, and bias-variance characteristics.

2. Dataset

The experiment uses the **Spambase dataset**, a benchmark binary classification dataset designed for spam email detection. The dataset consists of numerical features extracted from email content and a binary class label indicating whether an email is *spam* or *non-spam*.

Each instance in the dataset represents a single email described using frequency-based features, making it suitable for evaluating probabilistic and distance-based classifiers such as Naïve Bayes and K-Nearest Neighbors.

Dataset reference:

- Kaggle: Spambase Dataset

3. Preprocessing Steps

- The dataset file `spambase_csv_Kaggle.csv` was loaded into a Pandas DataFrame for analysis.
- The dataset was separated into the feature matrix (X), containing all numerical attributes, and the target vector (y), representing the spam and non-spam class labels.
- Feature scaling was performed using `StandardScaler` from the Scikit-learn library to normalize the input features, which is essential for distance-based algorithms such as KNN.
- The normalized data was split into training and testing sets using a standard train–test split strategy to evaluate model generalization performance.
- The resulting subsets X_{train}, X_{test}, y_{train}, and y_{test} were used for model training and evaluation.

4. Implementation Details

- Implemented Gaussian Naive Bayes, Bernoulli's Naive Bayes and Multinomial Naive Bayes.
- Implemented baseline K-Nearest Neighbors (KNN) Algorithm and with both KD-Tree and Ball-Tree Algorithm.
- Implemented hyperparameter tuning for KNN-Algorithm using the GridSearchCV & RandomizedSearchCV.
- Compared all the algorithm running time and its accuracy score and plotted the various plots to visualize the algorithm implementation.

5. Visualizations

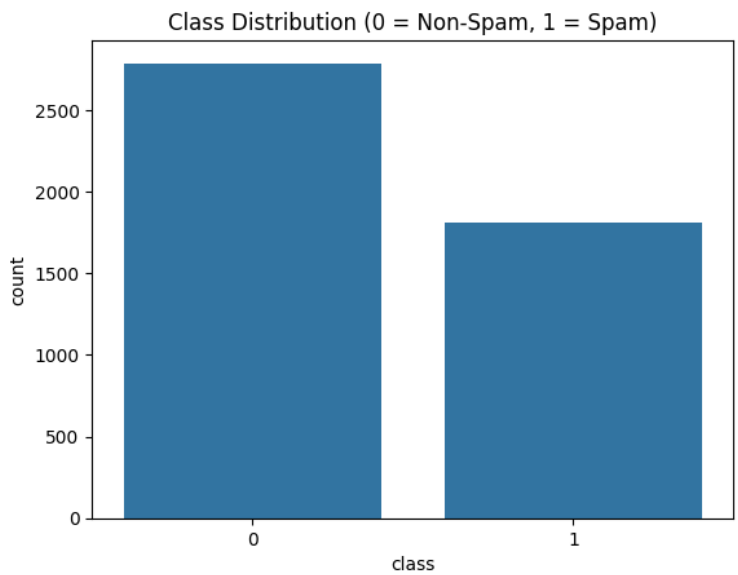

Figure 1: Class Distribution of Spam and Non-Spam Emails

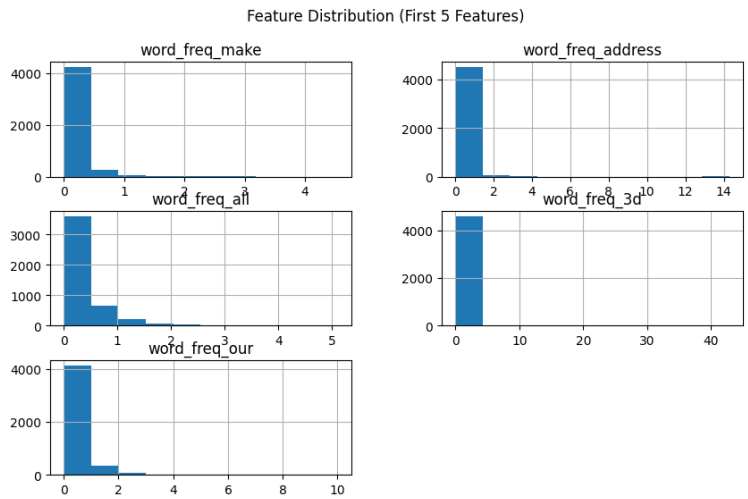

Figure 2: Feature Distribution of Selected Attributes

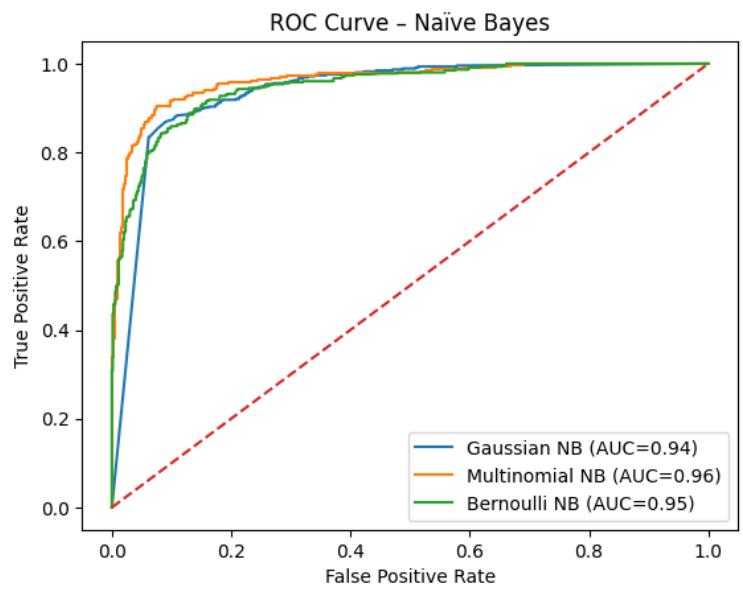

Figure 3: Confusion Matrix for Naïve Bayes Classifier

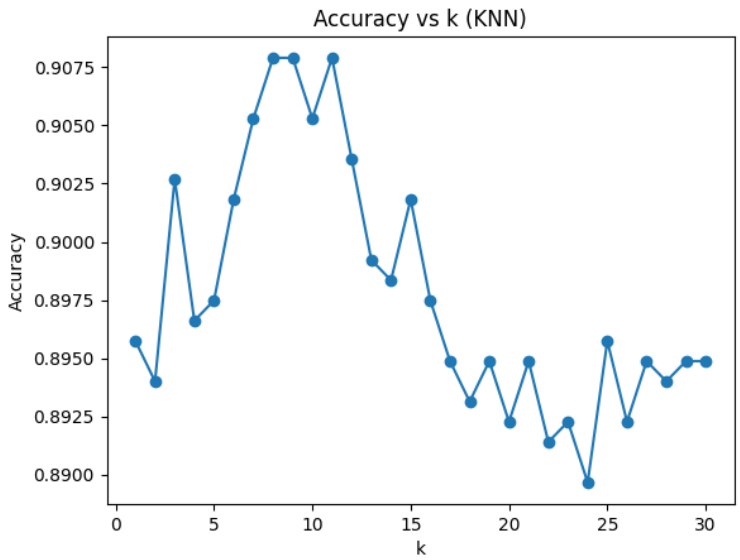

Figure 4: Confusion Matrix for K-Nearest Neighbors (KNN)

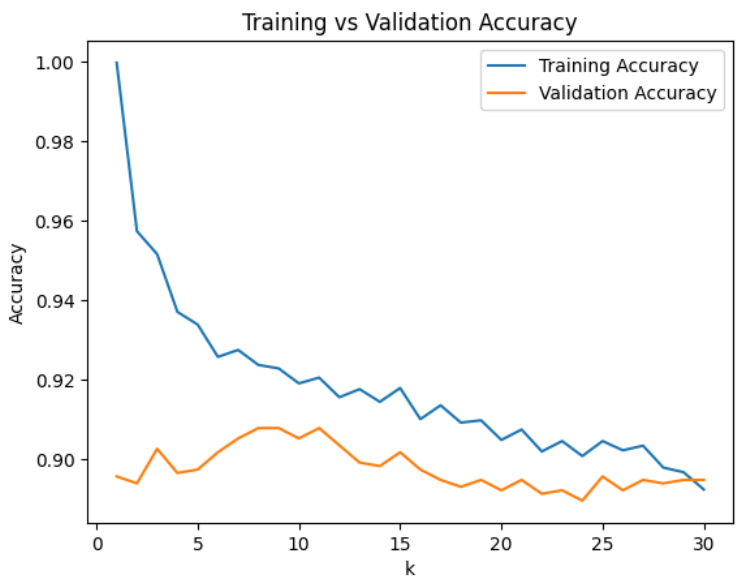

Figure 5: ROC Curve Comparison of Classifiers

6. Performance Comparison

Table 1: Naïve Bayes Performance Metrics

Metric	**Gaussian NB**	**Multinomial NB**	**Bernoulli NB**
Accuracy	0.825369	0.898349	0.883579
Precision	0.707718	0.951743	0.873832
Recall	0.949339	0.781938	0.823789
F1 Score	0.810913	0.858525	0.848073
Specificity	0.744620	0.974175	0.922525
False Positive Rate	0.255380	0.025825	0.077475
Training Time (s)	0.009084	0.005251	0.009940
Prediction Time (s)	0.002190	0.001041	0.001895

Table 2: KNN Hyperparameter Tuning

Search Method	**Best k**	**Best CV Accuracy**	**Best Parameters**
Grid Search	8	0.923188	metric: manhattan, n_neighbors: 8, weights: distance
Randomized Search	6	0.922319	metric: manhattan, n_neighbors: 6, weights: distance

Table 3: KNN Performance using BallTree

Metric	**Value**
Accuracy	0.923545
Precision	0.933649
Recall	0.867841
F1 Score	0.899543
Specificity	0.959828
False Positive Rate	0.040172
Training Time (s)	0.040814
Prediction Time (s)	0.472226

Table 4: KNN Performance using KDTree

Metric	**Value**
Accuracy	0.923545
Precision	0.933649
Recall	0.867841
F1 Score	0.899543
Specificity	0.959828
False Positive Rate	0.040172
Training Time (s)	0.049926
Prediction Time (s)	0.573689

Table 5: Comparison of Neighbor Search Algorithms

Criterion	**KDTree**	**BallTree**
Accuracy	0.923545	0.923545
Training Time (s)	0.049926	0.040814
Prediction Time (s)	0.573689	0.472226
Memory Usage	Low / Medium	Medium / High

7. Overfitting and Underfitting Analysis

- **Training vs. validation accuracy behavior in KNN**
  - Experimental results show that for smaller values of k, the KNN classifier achieves very high training accuracy but relatively lower validation accuracy.
  - This gap indicates overfitting, where the model memorizes training samples and becomes sensitive to noise.
  - As the value of k increases, the difference between training and validation accuracy reduces, indicating improved generalization.
  - For optimal values of k identified using cross-validation, both training and validation accuracies are stable and closely aligned.
  - When k becomes very large, both training and validation accuracies decrease, indicating underfitting due to overly smooth decision boundaries.
- **Effect of neighborhood size (k)**
  - **Small k values:**
    - High training accuracy and lower validation accuracy
    - High variance and sensitivity to outliers
    - Indicates overfitting
  - **Moderate k values (selected via tuning):**
    - Balanced training and validation accuracy
    - Best predictive performance on unseen test data
    - Represents an optimal bias–variance trade-off

- **Large k values:**
  - Reduced accuracy on both training and validation sets
  - Loss of local structure in the data
  - Indicates underfitting

- **Impact of hyperparameter tuning**
  - Grid Search and Randomized Search were used to select optimal values of k based on cross-validation accuracy.
  - Tuned KNN models demonstrated better generalization compared to manually selected k values.
  - This confirms that systematic hyperparameter tuning effectively reduces overfitting and underfitting.
  - Selecting k solely based on training accuracy would have resulted in inferior test performance.

8. Bias–Variance Analysis

- **Bias characteristics of Naïve Bayes**
  - Naïve Bayes classifiers assume conditional independence among features.
  - This assumption limits model flexibility, resulting in higher bias.
  - Experimental results show similar training and validation accuracies, indicating low variance.
  - Although stable, the model may slightly underfit when feature dependencies are present in the data.

- **Variance characteristics of KNN**
  - The variance of the KNN classifier is strongly influenced by the choice of k.
  - Small k values lead to low bias but high variance, as reflected by fluctuating validation accuracy.
  - Increasing k reduces variance and improves model stability.
  - Optimal k values obtained through tuning achieve a balanced bias–variance trade-off.

- **Effect of tuning on bias–variance trade-off**
  - Hyperparameter tuning adjusts model complexity by selecting appropriate neighborhood sizes.
  - Tuned KNN models reduce excessive variance while avoiding high bias.
  - This results in improved validation accuracy and consistent test performance.
  - The results highlight the importance of controlled tuning for achieving robust generalization.

9. Observation & Conclusion

In this experiment, multiple binary classification models were implemented and evaluated using appropriate training, validation, and test performance metrics. Naïve Bayes classifiers demonstrated stable and consistent performance with low variance due to their strong conditional independence assumption. However, this assumption resulted in higher bias, limiting their ability to model complex feature relationships present in the dataset.

The K-Nearest Neighbors (KNN) classifier exhibited greater flexibility and achieved higher accuracy when the neighborhood size was properly tuned. Experimental results showed that small values of k led to overfitting due to high variance, while very large values of k caused underfitting. Hyperparameter tuning using Grid Search and Randomized Search successfully identified optimal values of k, improving generalization performance and reducing overfitting.

Overall, the experiment highlights the importance of balancing bias and variance through appropriate model selection and hyperparameter tuning. Cross-validation played a crucial role in ensuring fair model comparison and selecting configurations that perform well on unseen data. to achieve robust generalization.

References

- Scikit-learn Documentation: Naïve Bayes Classifiers
- Scikit-learn Documentation: K-Nearest Neighbors
- Scikit-learn Documentation: Hyperparameter Optimization
- Kaggle: Spambase Dataset